Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

1. Tổng quan SOA & Microservices . . . 2
1.1. Kiến trúc phần mềm theo thời gian . . . 3
1.1.1. Thời kỳ Monolith . . . 3
1.1.2. Sự ra đời của SOA . . . 4
1.1.3. Từ SOA đến Microservices . . . 4
1.2. SOA là gì? Nguyên lý cốt lõi . . . 5
1.2.1. Bốn đặc trưng cốt lõi . . . 5
1.2.2. Tám nguyên lý hướng dịch vụ . . . 5
1.3. Microservices là gì? So sánh với SOA . . . 6
1.3.1. Định nghĩa và đặc trưng . . . 6
1.3.2. Scale Cube — ba chiều mở rộng . . . 7
1.3.3. So sánh SOA truyền thống vs Microservices . . . 7
1.4. Microservices trong thực tế — bài học từ các công ty công nghệ . . . 8
1.4.1. Netflix — khi downtime là không thể chấp nhận . . . 8
1.4.2. Amazon — API Mandate và “two-pizza teams” . . . 9
1.4.3. Uber — tổ chức lại microservices theo domain . . . 9
1.4.4. Bài học tổng hợp . . . 9
1.5. Đo lường và đánh giá hiệu năng kiến trúc . . . 10
1.5.1. Fast Flow Success Triangle — DevOps + Team Topologies + Architecture . . . 10
1.5.2. Đo lường Fast Flow — DORA Metrics . . . 12
1.5.3. Quality Attribute Scenarios — chỉ định yêu cầu kiến trúc . . . 12
1.6. Modular Monolith — lựa chọn hợp lệ . . . 13
1.6.1. Monolith không phải là xấu . . . 13
1.6.2. Khi nào Modular Monolith phù hợp? . . . 13
1.6.3. Con đường từ Monolith đến Microservices . . . 13
1.7. Các kịch bản áp dụng và hạn chế của Microservices . . . 14
1.7.1. Quyết định kiến trúc không chỉ là kỹ thuật . . . 14
1.7.2. Phân tích lợi ích và chi phí . . . 15
1.8. Case study: KBM . . . 16
1.8.1. Bài toán nghiệp vụ . . . 16
1.8.2. Điểm khởi đầu: những monolith rời rạc, không phải microservices . . . 17
1.8.3. Kiến trúc tổng quan . . . 17
1.9. Tổng kết . . . 21
1.10. Đọc thêm . . . 21
Tài liệu tham khảo . . . 22

CHƯƠNG 1

Tổng quan SOA & Microservices

> "Microservices are not the goal. You don't 'win' by having microservices."
> — Sam Newman, *Monolith to Microservices* [4b, Ch.2]

MỤC TIÊU HỌC TẬP

- Hiểu bối cảnh lịch sử dẫn đến kiến trúc hướng dịch vụ (SOA) và microservices
- Nắm được các nguyên lý cốt lõi của SOA và sự kế thừa trong microservices
- Phân biệt rõ SOA truyền thống, microservices, và modular monolith
- Tìm hiểu vì sao các công ty công nghệ lớn buộc phải chuyển đổi, và bài học rút ra
- Đánh giá khi nào nên — và khi nào *không* nên — áp dụng microservices
- Làm quen với case study của sách: KBM — một hệ thống LMS cho trường đại học

1.1. Kiến trúc phần mềm theo thời gian

Lịch sử phát triển phần mềm doanh nghiệp là câu chuyện về những lần vượt qua giới hạn: mỗi khi một hệ thống đạt đến quy mô mà kiến trúc hiện tại không còn đáp ứng được, cộng đồng lại tìm ra cách tiếp cận mới. Nhìn lại quá trình tiến hóa này giúp chúng ta hiểu *tại sao* microservices tồn tại, thay vì chỉ biết *nó là gì*.

1.1.1. Thời kỳ Monolith

Trong nhiều thập kỷ, hầu hết phần mềm được xây dựng theo kiến trúc **monolithic** (nguyên khối) — toàn bộ logic nghiệp vụ, giao diện người dùng, và truy cập dữ liệu nằm trong một ứng dụng duy nhất, triển khai như một đơn vị.

Kiến trúc monolith có những ưu điểm rõ ràng. Một codebase duy nhất giúp phát triển đơn giản: mở IDE, chạy ứng dụng, gỡ lỗi (debug) với các breakpoint — mọi thứ liền mạch. Việc triển khai (deploy) chỉ cần một artifact và một câu lệnh. Hiệu năng tốt vì gọi hàm cục bộ (in-process) luôn nhanh hơn gọi qua mạng nhiều bậc. Và quan trọng nhất, khi mọi thao tác dùng chung một database, transaction ACID cục bộ giúp giữ dữ liệu nhất quán đơn giản hơn nhiều — dù ràng buộc nghiệp vụ và mức cô lập vẫn phải được thiết kế đúng.

Tuy nhiên, khi hệ thống phát triển — thêm tính năng, thêm thành viên, thêm khách hàng — monolith bắt đầu bộc lộ những giới hạn nghiêm trọng. Thomas Erl gọi đây là kiến trúc dạng silo (*silo-based application architecture*) — một trạng thái mà mỗi ứng dụng trở thành "ốc đảo" với logic trùng lặp, khó tích hợp.

Monolith và tòa nhà đa năng

Chúng ta có thể hình dung kiến trúc Monolith giống như một tòa nhà đa năng duy nhất của trường đại học, nơi chứa cả giảng đường, thư viện, nhà ăn và phòng thí nghiệm. Ở quy mô nhỏ, sinh viên chỉ cần đi vài bước là đến nơi, quản lý tòa nhà cũng dễ dàng. Nhưng khi số lượng sinh viên tăng lên hàng chục nghìn, việc sửa đường ống nước ở nhà ăn có thể phải đóng cửa luôn cả thư viện và giảng đường, gây gián đoạn toàn bộ hoạt động học tập.

Các biểu hiện cụ thể:

Bảng 1.1: *Các biểu hiện khi monolith đạt đến giới hạn*

Biểu hiện	Mô tả	Hệ quả
Nút thắt phối hợp	Mỗi thay đổi nhỏ đòi hỏi phối hợp nhiều team	Tốc độ phát triển giảm dần theo kích thước team
Ràng buộc triển khai	Một tính năng mới đòi hỏi phải triển khai (deploy) lại toàn bộ	Chu kỳ phát hành kéo dài, rủi ro cao
Ràng buộc công nghệ	Toàn ứng dụng dùng chung stack	Không thể thử nghiệm công nghệ mới cho một phần
Hạn chế mở rộng cục bộ	Mở rộng = nhân bản toàn bộ ứng dụng	Chi phí cao, lãng phí tài nguyên
Quá tải độ phức tạp	Codebase quá lớn, ranh giới module mờ nhạt	Lập trình viên mới mất hàng tuần, hàng tháng để hiểu hệ thống

Khi monolith đạt đến trạng thái mà không ai hiểu toàn bộ, mỗi thay đổi đều tiềm ẩn hệ quả ngoài ý muốn (side effect), và chu kỳ phát hành (release) kéo dài từ tuần sang tháng — đó chính là trạng thái mà Chris Richardson gọi là **"monolithic hell"**.

1.1.2. Sự ra đời của SOA

Đầu thập niên 2000, ngành công nghiệp phần mềm nhận ra rằng vấn đề không nằm ở quy mô ứng dụng, mà ở *cách tổ chức* logic bên trong. *Service-Oriented Architecture* (SOA — kiến trúc hướng dịch vụ) ra đời với ý tưởng cốt lõi: **phân rã hệ thống thành các dịch vụ có thể tái sử dụng**, giao tiếp qua giao thức chuẩn.

SOA hướng tới ba mục tiêu lớn: tăng khả năng tái sử dụng logic nghiệp vụ trên toàn doanh nghiệp, giảm tổng lượng code trùng lặp giữa các hệ thống, và cải thiện khả năng tích hợp giữa các nền tảng khác nhau.

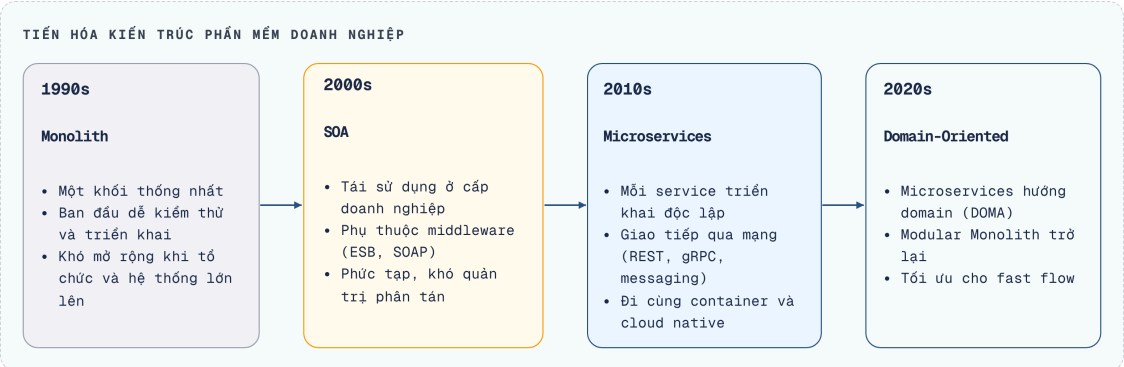
Tuy nhiên, SOA truyền thống thường đi kèm với Enterprise Service Bus (ESB) — một middleware trung tâm điều phối mọi giao tiếp. ESB nhanh chóng trở thành điểm nghẽn: phức tạp, nặng nề, và tạo ra một dạng coupling mới — *coupling vào middleware*. Đây là bài học mà microservices rút ra để thiết kế theo nguyên tắc ngược: "smart endpoints, dumb pipes".

ESB ≠ API Gateway

Đừng nhầm ESB với API Gateway hiện đại. Gateway xử lý các mối quan tâm hạ tầng — định tuyến, xác thực, rate limiting — theo đúng tinh thần "dumb pipes": không chứa business logic; còn ESB ôm cả business logic và orchestration phức tạp vào middleware. Dù vậy, vẫn cần cảnh giác: nếu lập trình viên dồn business logic vào Gateway, nó sẽ dần biến thành một "ESB kiểu mới" (Chương 8).

1.1.3. Từ SOA đến Microservices

Khoảng năm 2014, James Lewis và Martin Fowler đăng bài viết "Microservices" trên martinfowler.com, phổ biến rộng rãi thuật ngữ **microservices** để mô tả một phong cách kiến trúc đã tiến hóa từ SOA nhưng với mục tiêu khác biệt (thuật ngữ này được sử dụng trước đó tại các workshop phần mềm từ ~2011). Microservices không phải SOA phiên bản nhỏ — đây là sự chuyển dịch căn bản từ **tái sử dụng** (reuse) sang **tính độc lập** (independence) như ưu tiên số một.



Hình 1.1: *Tiến hóa kiến trúc phần mềm doanh nghiệp từ thập niên 1990 đến 2020*

1.2. SOA là gì? Nguyên lý cốt lõi

Dù microservices đã trở thành xu hướng chính, việc hiểu SOA vẫn quan trọng vì hai lý do: thứ nhất, nhiều doanh nghiệp lớn vẫn vận hành hệ thống SOA; thứ hai, các nguyên lý hướng dịch vụ của SOA là **nền tảng** mà microservices kế thừa và phát triển.

1.2.1. Bốn đặc trưng cốt lõi

Thomas Erl xác định SOA qua bốn đặc trưng cơ bản:

Business-Driven. Dịch vụ được thiết kế từ quy trình nghiệp vụ, không phải từ bảng cơ sở dữ liệu hay module kỹ thuật. Một dịch vụ "Đăng ký môn học" phản ánh quy trình đào tạo của sinh viên, không phải bảng `registrations` trong database. Nguyên tắc này vẫn giữ nguyên giá trị trong microservices — và trở thành nền tảng cho Domain-Driven Design (DDD) mà chúng ta sẽ khám phá trong Chương 2.

Vendor-Neutral. Dịch vụ giao tiếp qua giao thức chuẩn (HTTP, messaging), không phụ thuộc vào công nghệ cụ thể. Đây là điều kiện tiên quyết cho **technology diversity** — một trong những lợi thế lớn nhất của microservices.

Enterprise-Centric. SOA tư duy ở cấp toàn tổ chức: dịch vụ không phục vụ riêng một ứng dụng mà phục vụ toàn doanh nghiệp (giống như một "Phòng Đào tạo" chung phục vụ mọi khoa trong trường đại học). Microservices thu hẹp phạm vi này vào từng *bounded context* — nhưng vẫn giữ tư duy dịch vụ vượt ra ngoài một team duy nhất.

Composition-Centric. Dịch vụ được thiết kế để tổ hợp (compose) thành workflow phức tạp. Trong microservices, điều này thể hiện qua *Saga pattern* (giống như quy trình đăng ký tín chỉ: nếu hệ thống đã giữ chỗ lớp học và xuất hóa đơn, nhưng sinh viên không nộp học phí đúng hạn, hệ thống phải lần lượt gọi ngược lại các dịch vụ trước đó để 'hủy hóa đơn' và 'giải phóng chỗ ngồi') và *API Gateway* — các chủ đề của Chương 6 và 8.

1.2.2. Tám nguyên lý hướng dịch vụ

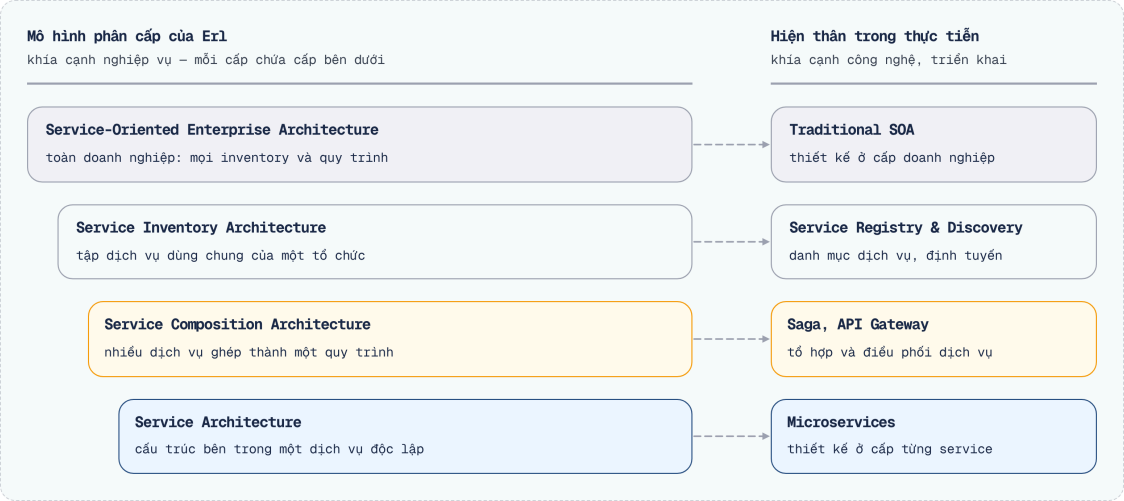
Erl đúc kết 8 nguyên lý nền tảng cho mọi dịch vụ. Bảng 1.2 cho thấy microservices kế thừa và chuyển hóa chúng như thế nào:

Bảng 1.2: *Tám nguyên lý hướng dịch vụ — SOA truyền thống vs Microservices*

#	Nguyên lý	Trong SOA truyền thống	Trong Microservices
1	**Standardized Contract**	WSDL, XML Schema	OpenAPI/Swagger, Protocol Buffers
2	**Loose Coupling**	ESB giảm coupling	**Ưu tiên số 1**: database-per-service, async messaging
3	**Abstraction**	Interface ẩn implementation	API là contract duy nhất, code là black box
4	**Reusability**	**Mục tiêu chính** — tái sử dụng ở cấp enterprise	Giảm ưu tiên — *independence* quan trọng hơn *reuse*
5	**Autonomy**	Hạn chế bởi shared infrastructure	**Mục tiêu chính** — own database, own lifecycle
6	**Statelessness**	Stateless design	RESTful by default, state externalized (Redis, DB)
7	**Discoverability**	UDDI registry	Service Discovery (Eureka, Consul, K8s DNS)
8	**Composability**	ESB orchestration	Saga, API Gateway, event choreography

Điểm mấu chốt: microservices không phủ nhận SOA — mà **chọn lọc và đẩy mạnh** những nguyên lý phù hợp (Loose Coupling, Autonomy) và giảm nhẹ những cơ chế dễ tạo liên kết chặt ở quy mô toàn doanh nghiệp (Reusability ở cấp enterprise, kho đặc tả tập trung kiểu UDDI và điều phối qua ESB).

Bốn kiểu kiến trúc SOA. SOA tồn tại ở nhiều cấp độ, từ nhỏ đến lớn:



__Hình 1.2:__ Bốn kiểu kiến trúc SOA — từ dịch vụ đơn lẻ đến quy mô doanh nghiệp

Hầu hết dự án microservices thực tế hoạt động ở tầng **Service Architecture** và **Service Composition Architecture**. Sự khác biệt cốt lõi: SOA truyền thống cố gắng quản lý một "Service Inventory" thống nhất cho toàn doanh nghiệp — trong khi microservices chấp nhận sự phân tán và trao quyền cho từng team.

1.3. Microservices là gì? So sánh với SOA

Có thể hình dung microservices qua cách một trường đại học tự tổ chức: thay vì một phòng ban duy nhất lo toàn bộ từ tuyển sinh, đào tạo đến cấp bằng, trường chia thành các khoa và phòng ban hoạt động độc lập — mỗi đơn vị tự quyết định cách vận hành và lưu trữ hồ sơ riêng.

1.3.1. Định nghĩa và đặc trưng

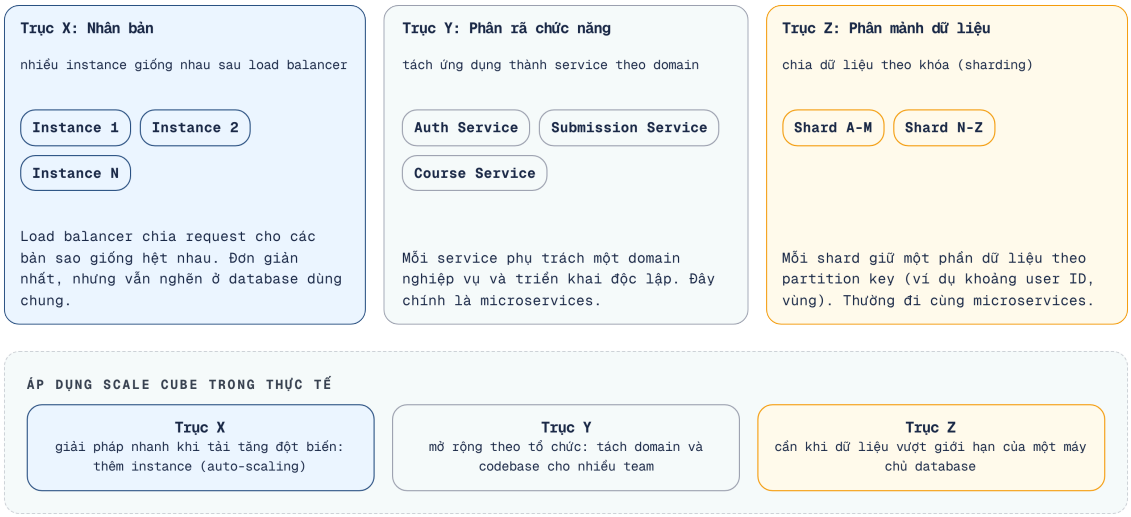
Microservices là phong cách kiến trúc trong đó ứng dụng được cấu thành từ **tập hợp các dịch vụ nhỏ, tự trị**. Mỗi dịch vụ thỏa mãn bốn điều kiện:

1. **Chạy trong tiến trình riêng** — deploy và restart độc lập
2. **Sở hữu dữ liệu riêng** — database-per-service, không chia sẻ bảng (lưu ý: có thể chỉ là các logical schemas riêng biệt trên cùng một DB server vật lý để tối ưu chi phí)
3. **Giao tiếp qua giao thức nhẹ** — HTTP/REST, gRPC, hoặc messaging (Kafka, RabbitMQ)
4. **Có thể triển khai độc lập** — thay đổi service A không yêu cầu triển khai lại service B

Đặc biệt, điều kiện thứ tư (*independent deployability*) chính là thước đo quan trọng nhất. Nếu không thể deploy một service mà không phải deploy các service khác — đó chưa phải microservices thực sự.

1.3.2. Scale Cube — ba chiều mở rộng

Để hiểu microservices trong bối cảnh scalability, chúng ta sử dụng mô hình **Scale Cube** do Martin L. Abbott và Michael T. Fisher giới thiệu trong cuốn "The Art of Scalability", sau đó được Chris Richardson phổ biến trong ngữ cảnh microservices. Mô hình này chia mở rộng hệ thống theo ba chiều:



Hình 1.3: Scale Cube — ba chiều mở rộng hệ thống

- **Trục X — Nhân bản**: chạy nhiều instance giống nhau phía sau load balancer. Đơn giản nhất, giới hạn bởi database bottleneck
- **Trục Y — Phân rã chức năng**: tách ứng dụng thành các service theo domain. **Đây là microservices**
- **Trục Z — Phân mảnh dữ liệu**: chia dữ liệu theo tiêu chí (region, user ID). Thường kết hợp với microservices

Scaling truyền thống dừng ở hai lựa chọn: nâng cấp phần cứng cho một máy (tăng RAM/CPU — *vertical scaling*, nằm ngoài Scale Cube) và nhân bản instance phía sau load balancer (trục X). Khi cả hai đạt giới hạn — database trở thành bottleneck, deployment trở nên rủi ro — đó là lúc cần xem xét trục Y.

1.3.3. So sánh SOA truyền thống vs Microservices

Bảng 1.3 so sánh hai cách tiếp cận theo tám khía cạnh, từ phạm vi và kích thước dịch vụ đến mô hình tổ chức đội.

Bảng 1.3: So sánh SOA truyền thống và Microservices

Khía cạnh	SOA truyền thống	Microservices
Phạm vi	Toàn doanh nghiệp	Một ứng dụng hoặc bounded context
Kích thước dịch vụ	Lớn, tái sử dụng tối đa	Nhỏ, focused, single responsibility
Giao tiếp	ESB — middleware trung tâm	"Smart endpoints, dumb pipes" — REST/messaging
Dữ liệu	Thường shared database	Database-per-service
Governance	Centralized, formal	Decentralized, team-owned
Trọng tâm thiết kế	Reuse-first	Independence-first

Khía cạnh	SOA truyền thống	Microservices
Deploy	Thường cùng deploy	Independent deployment
Team model	Chuyên môn theo layer (DB team, UI team)	Cross-functional, own the service

Cần lưu ý rằng SOA, microservices, và event-driven architecture (EDA) không phải các lựa chọn loại trừ nhau. Chúng là các lớp của một cách tiếp cận đang tiến hóa: SOA cung cấp nguyên lý nền tảng, microservices hiện thực hóa chúng ở mức độ phân rã chi tiết hơn (fine-grained), và EDA bổ sung khả năng giao tiếp bất đồng bộ cho các hệ thống phân tán phức tạp. Chúng ta sẽ khám phá EDA chi tiết trong Chương 5.

1.4. Microservices trong thực tế — bài học từ các công ty công nghệ

Lý thuyết về microservices chỉ thực sự trở nên thuyết phục khi nhìn vào *tại sao* các tổ chức lớn buộc phải chuyển đổi. Không phải vì microservices "thời thượng" — mà vì họ đối mặt với **những giới hạn mà kiến trúc truyền thống và cách mở rộng theo trục X không thể vượt qua**.

1.4.1. Netflix — khi downtime là không thể chấp nhận

Năm 2008, Netflix gặp sự cố database corruption nghiêm trọng, gây gián đoạn dịch vụ. Sự cố này là chất xúc tác cho quyết định chuyển toàn bộ hạ tầng lên cloud (AWS) và tái cấu trúc sang microservices. Theo Netflix Technology Blog [W3], Netflix hoàn thành chuyển đổi hạ tầng sang AWS vào đầu 2016 — đây là cột mốc của quá trình dịch chuyển (migration) lên đám mây, nhưng kiến trúc microservices tiếp tục tiến hóa qua nhiều năm sau đó. Tại thời điểm 2016, hệ thống vận hành hàng trăm microservices phục vụ gần 100 triệu người dùng (Netflix vượt mốc 100 triệu thuê bao vào giữa năm 2017).

Vấn đề cốt lõi là đòi hỏi khắt khe về tính sẵn sàng (uptime) và khả năng mở rộng (scale). Với lượng người dùng tăng theo cấp số nhân, một monolith duy nhất trở thành single point of failure. Vertical scaling (nâng cấp server) đạt giới hạn vật lý và chi phí. Netflix cần hai thứ mà monolith không thể cung cấp: khả năng mở rộng từng thành phần riêng biệt, và **fault isolation** — lỗi ở hệ thống recommendation không được phép ảnh hưởng đến streaming.

Fault Isolation — cách ly sự cố trong mùa thi

Tương tự như việc tổ chức thi học kỳ tập trung: Nếu tất cả sinh viên thi cùng một phòng lớn (monolith), một sự cố như mất điện hay hỏng máy chiếu sẽ ảnh hưởng tới toàn bộ thí sinh. Với Microservices, sinh viên được chia vào nhiều phòng thi nhỏ biệt lập (fault isolation). Phòng thi số 1 có thể bị hỏng điều hòa, nhưng thí sinh ở các phòng khác vẫn làm bài bình thường. Hệ thống gợi ý phim (recommendation) của Netflix gặp sự cố thì người dùng vẫn phải có khả năng xem phim (streaming).

Netflix đóng góp ngược lại cho cộng đồng bằng các công cụ mã nguồn mở: **Eureka** (Service Discovery), **Zuul** (API Gateway), **Hystrix** (Circuit Breaker) — nhiều trong số đó trở thành nền tảng của Spring Cloud. Zuul 1 và Hystrix nay đã ở chế độ bảo trì, được thay bằng Spring Cloud Gateway (KBM đang dùng) và Resilience4j (nằm trong lộ trình của KBM, Chương 4 và 10); Eureka vẫn được dùng.

Sự ra đời của Docker vào năm 2013 cũng là một "chất xúc tác" (enabler) quan trọng giúp kiến trúc microservices thực sự bùng nổ, vì nó giải quyết bài toán vận hành hàng chục tiến trình độc lập một cách đồng nhất và dễ dàng.

1.4.2. Amazon — API Mandate và "two-pizza teams"

Khoảng 2001–2002, Amazon đối mặt với tình trạng monolith liên kết quá chặt chẽ (tightly coupled) — tầng giao diện (frontend) và tầng xử lý (backend) phụ thuộc vào nhau đến mức *bất kỳ thay đổi nào, dù nhỏ, đều ảnh hưởng toàn hệ thống*. Theo câu chuyện nổi tiếng được Steve Yegge kể lại trong "Google Platforms Rant" (2011) [W5], Jeff Bezos được cho là đã ra **"API mandate"** — một chỉ thị nội bộ: tất cả team phải giao tiếp với nhau *chỉ* qua service interfaces, không có ngoại lệ. Cùng giai đoạn, Amazon tổ chức thành các nhóm đủ nhỏ để hai chiếc pizza là đủ cho cả nhóm ăn (two-pizza teams), mỗi nhóm sở hữu trọn dịch vụ của mình.

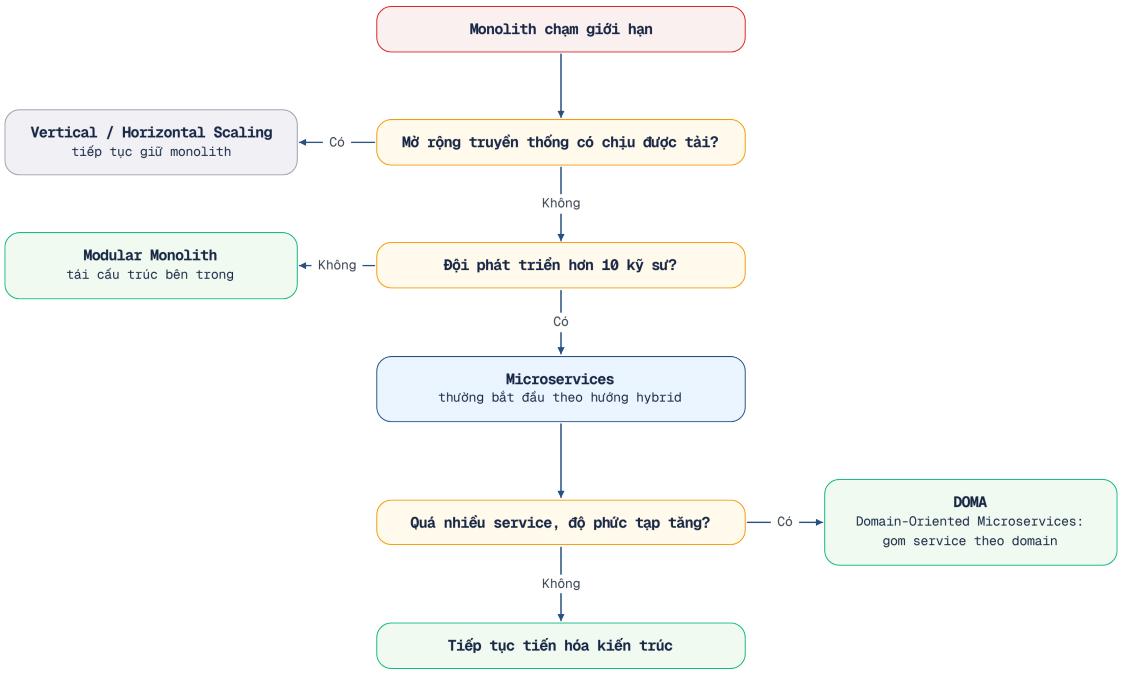
1.4.3. Uber — tổ chức lại microservices theo domain

Uber bắt đầu như một monolith Python đơn giản, tăng trưởng với tốc độ bùng nổ, và khoảng 2014–2016 chuyển sang microservices. Tuy nhiên, câu chuyện Uber đặc biệt đáng chú ý ở phần *hậu chuyển đổi*: đến khoảng 2020, với hơn 2.000 microservices, họ nhận ra hệ thống đang rơi vào *"microservice madness"* — quá nhiều service nhỏ tạo ra mạng lưới phụ thuộc phức tạp, khó debug và vận hành.

Uber phản ứng bằng cách giới thiệu **DOMA** (*Domain-Oriented Microservice Architecture*) — nhóm các microservices liên quan thành "domain" lớn hơn, giảm bề mặt tương tác. Uber không quay về monolith mà tổ chức lại: các microservice được gom theo domain, có quy tắc phụ thuộc giữa các tầng và gateway ở biên mỗi domain.

1.4.4. Bài học tổng hợp

Hình 1.4 tóm tắt ba câu chuyện trên thành một cây quyết định chuyển đổi kiến trúc.



Hình 1.4: *Cây quyết định chuyển đổi kiến trúc — từ monolith đến microservices*

Ba pattern chung nổi lên:

1. **Chuyển đổi vì bị buộc, không vì mong muốn.** Mỗi công ty trên đều chuyển sang microservices khi cách mở rộng truyền thống thất bại trước yêu cầu nghiêm ngặt — uptime 99.99%, hàng triệu request/giây, hàng trăm lập trình viên cần triển khai độc lập.
2. **Cấu trúc nhóm thay đổi trước (hoặc cùng) kiến trúc.** Amazon chia thành các nhóm nhỏ (two-pizza teams) trước khi tách service. Netflix tạo ra các "full-cycle developers" — mỗi lập trình viên chịu trách nhiệm từ khâu viết mã đến khi vận hành trên production. Kiến trúc phần mềm phản ánh cấu trúc tổ chức (Conway's Law). Ngành công nghiệp thậm chí còn áp dụng "Inverse Conway Maneuver" (Nghịch đảo định luật Conway) — chủ động thay đổi cấu trúc team để ép kiến trúc phần mềm phải thay đổi theo. Chúng ta sẽ phân tích sâu trong Chương 2.
3. **Kiến trúc còn tiếp tục tiến hóa sau chuyển đổi.** Microservices không phải đích cuối: Uber phải tổ chức lại hơn 2.000 service thành domain (DOMA), và nhiều tổ chức vẫn giữ monolith hoặc modular monolith cho những phần chưa cần tách — kết hợp như vậy là hợp lý, không phải thất bại.

Fast Flow

"The primary motivation for using the microservice architecture is to enable fast flow — the continuous delivery of a stream of small changes with rapid feedback on each one."

— Chris Richardson, Microservices Patterns, 2nd Ed.

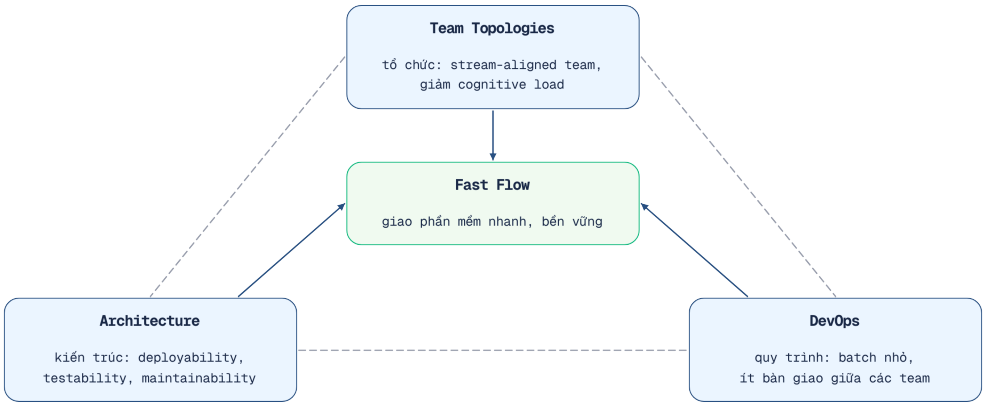
1.5. Đo lường và đánh giá hiệu năng kiến trúc

1.5.1. Fast Flow Success Triangle — DevOps + Team Topologies + Architecture

Phần nâng cao (có thể lướt qua)

Ba mục tiếp theo (*Fast Flow Success Triangle*, *DORA Metrics*, *Quality Attribute Scenarios*) là góc nhìn **đo lường và đánh giá** kiến trúc — hữu ích nhưng không bắt buộc để hiểu microservices là gì. Nếu đây là lần đọc đầu, người đọc có thể đọc lướt và quay lại sau: DORA Metrics sẽ gặp lại trong ngữ cảnh CI/CD ở **Chương 12**, còn Quality Attribute Scenarios sẽ phát huy tác dụng khi cân nhắc tách/gộp service ở **Chương 2**. Một số thuật ngữ xuất hiện sớm trong chương này (CAP, BASE, eventual consistency, Saga...) sẽ được giải thích kỹ ở các chương sau; có thể tra nhanh ở **Phụ lục A — Bảng thuật ngữ** bất cứ lúc nào.

Giống như việc nhà trường cần phối hợp nhịp nhàng giữa Phòng Đào tạo (quy trình), đội ngũ giảng viên (tổ chức) và cơ sở vật chất (kiến trúc) để một học kỳ diễn ra suôn sẻ, việc phát triển phần mềm cũng đòi hỏi sự đồng bộ tương tự. Richardson trong phiên bản thứ hai (2025) giới thiệu một framework quan trọng: **Fast Flow Success Triangle** — mô tả ba yếu tố cần thiết để đạt được fast flow:



Hình 1.5: *Fast Flow Success Triangle — ba yếu tố cần thiết cho fast flow*

DevOps (Quy trình) — Không phải tên team hay job title, mà là tập hợp nguyên tắc và thực hành để phân phối phần mềm nhanh và tin cậy. Hai nguyên tắc quan trọng nhất: (1) *chia nhỏ khối lượng mỗi lần cập nhật (batch size)* — commit và push các thay đổi nhỏ, lý tưởng ít nhất 1 lần/ngày; (2) *giảm các khâu bàn giao chéo giữa các team* — mỗi bước bàn giao đều gây ra độ trễ và rơi rớt thông tin.

Team Topologies (Tổ chức) — Cấu trúc tổ chức theo bốn kiểu team (stream-aligned, platform, enabling, complicated subsystem) với ba kiểu tương tác (X-aaS, collaboration, facilitating). Stream-aligned teams làm phần lớn công việc, các team khác hỗ trợ.

Architecture (Kiến trúc) — Mục đích của kiến trúc, ít nhất từ góc độ fast flow, là **tạo tiền đề cho** DevOps và Team Topologies. Richardson trong [2b, Ch.5] phân tích các **architecture qualities** (thuộc tính kiến trúc) phục vụ fast flow; với bối cảnh KBM, chương này tập trung vào ba thuộc tính cốt lõi nhất:

- **Deployability (Khả năng triển khai)**: Mỗi service có thể được triển khai độc lập với pipeline riêng, không cần phối hợp với các service khác. Ví dụ tại KBM: Judge Service có thể được triển khai mà Core Service không cần biết.
- **Testability (Khả năng kiểm thử)**: Có thể kiểm thử một service hoàn toàn cô lập, không cần khởi chạy toàn bộ hệ thống. Ví dụ: kiểm thử logic của Judge mà không cần chạy Gateway, Auth, hay Kafka.
- **Maintainability (Khả năng bảo trì)**: Các nhóm có thể thay đổi mã nguồn dễ dàng, ít xung đột, với quyền sở hữu (ownership) rõ ràng. Giả sử khi mở rộng thành nhiều nhóm: Nhóm A sở hữu Judge, Nhóm B sở hữu Academic, cả hai có thể phát triển song song mà không chặn lẫn nhau.

Ba thuộc tính này là *điều kiện cần* cho fast flow — thiếu bất kỳ thuộc tính nào sẽ tạo điểm nghẽn (bottleneck). KBM hiện chưa đạt trọn vẹn cả ba: shared library thường kéo theo việc biên dịch lại nhiều service (deployability), shared database gây phụ thuộc khi kiểm thử tích hợp (testability), và các "god class" (lớp ôm đồm quá nhiều chức năng) hơn 1.000 dòng làm tăng nguy cơ xung đột mã nguồn và khó bảo trì (maintainability).

Tam giác chỉ đứng vững khi đủ cả ba chân. DevOps tốt và team tốt nhưng kiến trúc monolith coupling chặt thì fast flow vẫn không thể đạt được. Ngược lại, microservices architecture tốt nhưng team vẫn tổ chức theo layer (DB team, UI team) thì lợi ích sẽ bị triệt tiêu bởi coordination overhead.

1.5.2. Đo lường Fast Flow — DORA Metrics

Làm sao biết chúng ta đã đạt fast flow? Nghiên cứu của Forsgren, Humble và Kim trong *Accelerate* (2018) định nghĩa bốn chỉ số DORA (DevOps Research and Assessment):

Bảng 1.4: *DORA Metrics — so sánh Elite vs Low Performers*

Metric	Elite Performers	Low Performers	Chênh lệch
Deployment Frequency	Theo nhu cầu (nhiều lần/ngày)	1 lần/1–6 tháng	208×
Lead Time (commit → production)	< 1 ngày	1–6 tháng	106×
Change Failure Rate	0–15%	46–60%	7×
Recovery Time (MTTR)	< 1 giờ	1 tuần–1 tháng	2.604×

Nguồn: DORA State of DevOps Report 2019 (bộ chỉ số định nghĩa trong Accelerate; con số cụ thể thay đổi qua từng năm)

Điều quan trọng hơn cả: **triển khai thường xuyên hơn lại *tin cậy* hơn**. Elite performers deploy với tần suất cao hơn 208 lần nhưng tỷ lệ lỗi thấp hơn 7 lần. Các thay đổi nhỏ sẽ dễ kiểm thử, debug và rollback hơn — nguyên lý "move fast and *don't* break things."

DORA Metrics cho LMS

KBM hiện tại:

- **Deployment Frequency**: Thủ công, khi cần; chưa đo tần suất nên chưa xếp được nhóm DORA
- **Lead Time**: Chưa đo, nhưng thiếu CI/CD tự động → ước tính ngày–tuần
- **Change Failure Rate**: Chưa theo dõi → chưa có số liệu
- **Recovery Time**: Khởi động lại thủ công → ước tính phút–giờ (mức chấp nhận được trong môi trường học thuật)

Lộ trình chuyển đổi (Migration path): Triển khai CI/CD pipeline (Chương 12) là bước đầu tiên để cải thiện DORA Metrics. Mục tiêu: deploy hàng tuần và lead time < 1 tháng — tương ứng nhóm Medium theo phân cụm DORA 2019; chạm mốc lead time < 1 tuần là đã sang ngưỡng High.

1.5.3. Quality Attribute Scenarios — chỉ định yêu cầu kiến trúc

DORA Metrics đo lường kết quả cuối cùng, nhưng làm sao chỉ định kiến trúc phải hỗ trợ kết quả đó ngay từ khâu thiết kế? Richardson trong [2b, Ch.5] khuyên dùng **Quality Attribute Scenarios** — kỹ thuật do Software Engineering Institute (SEI) phát triển, cho phép viết "spec" cho các thuộc tính phi chức năng một cách đo lường được.

Thay vì nói chung chung "hệ thống phải dễ deploy", một kịch bản chất lượng định nghĩa rõ ràng theo khuôn: **Source → Stimulus → Artifact → Environment → Response → Response Measure**

Ví dụ — kịch bản deployability cho LMS:

- **Source**: Lập trình viên (stream-aligned team)
- **Stimulus**: Commit code thay đổi logic chấm điểm SQL
- **Artifact**: Judge Service
- **Environment**: Deployment pipeline
- **Response**: Pipeline tự động chạy CI, build, test, và triển khai phiên bản mới lên production — mà không cần sự can thiệp của nhóm khác
- **Response Measure**: Lead time < 40 phút

Ví dụ — kịch bản scalability cho LMS:

- **Source**: 200 sinh viên thi cùng lúc (chế độ thi)
- **Stimulus**: Lượng bài nộp tăng vọt lên hàng trăm lượt mỗi phút trong những phút cuối giờ (sinh viên nộp lại nhiều lần)
- **Artifact**: Judge Service cluster
- **Environment**: Vận hành bình thường trong ca thi, cao điểm ở những phút cuối giờ
- **Response**: Tất cả bài nộp được xử lý thành công, không yêu cầu nào bị từ chối
- **Response Measure**: p99 thời gian *tiếp nhận* bài nộp (enqueue) < 500 ms; p99 thời gian *trả kết quả chấm* < 45 giây (bản thân việc chấm mất 5–30 giây, xem §4.1); 0% message loss

Bằng cách viết ra các kịch bản này, chúng ta có một dạng "unit test cho kiến trúc" — bất kỳ quyết định tách hay gộp service nào đều phải đối chiếu lại: *"Quyết định này có giúp chúng ta đạt kịch bản đó hay không?"*

1.6. Modular Monolith — lựa chọn hợp lệ

Trước khi nghĩ đến microservices, chúng ta cần dành thời gian nghiêm túc cho một lựa chọn thường bị bỏ qua: **Modular Monolith**. Đây có thể là bước đi hợp lý nhất, tùy ngữ cảnh.

1.6.1. Monolith không phải là xấu

Không phải mọi monolith đều giống nhau. Theo mức độ tổ chức nội bộ, có thể phân biệt hai loại:

- **Patchwork Monolith (Monolith chắp vá)**: Đặc trưng bởi mã nguồn lộn xộn, thiếu cấu trúc, mọi class phụ thuộc lẫn nhau, ranh giới mờ nhạt, thường được gọi là "big ball of mud". Hệ thống này cần được refactor nghiêm túc để xác định lại cả ranh giới bên trong lẫn bên ngoài.
- **Modular Monolith (Monolith theo module)**: Hệ thống có các module phân chia rõ ràng, giao tiếp với nhau qua các interface được định nghĩa sẵn. Mỗi module thậm chí có thể có schema riêng trong cùng một database. Mặc dù phải deploy cùng nhau, các module có thể được phát triển tương đối độc lập và hoàn toàn **có thể tách dần** ra thành các service riêng biệt khi quy mô đòi hỏi.

Monolith ≠ Bad

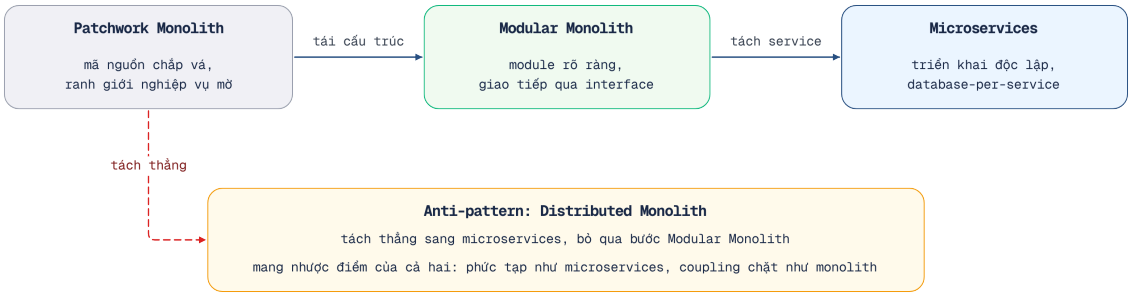
Monolith phản ánh kiến trúc đóng gói, không phản ánh chất lượng. Một modular monolith được thiết kế tốt hoàn toàn là lựa chọn kiến trúc hợp lệ — đặc biệt khi chi phí vận hành microservices vượt quá lợi ích mà nó mang lại.

1.6.2. Khi nào Modular Monolith phù hợp?

- **Team nhỏ**: dưới khoảng 5 người, overhead vận hành microservices (CI/CD per service, monitoring, distributed debugging) thường áp đảo lợi ích; với 5–10 người, modular monolith thường vẫn là lựa chọn hợp lý
- **Domain chưa rõ ràng**: Chia service quá sớm khi chưa hiểu domain rõ ràng dẫn đến chia sai. Gộp lại các service rất tốn kém và rủi ro so với refactor module trong monolith
- **Giai đoạn startup/MVP**: Cần tốc độ phát triển, đưa sản phẩm ra thị trường nhanh
- **Không cần polyglot**: Nếu toàn team dùng cùng ngôn ngữ/framework, lợi ích technology diversity không tồn tại

1.6.3. Con đường từ Monolith đến Microservices

Hình 1.6 mô tả con đường từ một patchwork monolith đến microservices, qua bước modular hóa.



Hình 1.6: Con đường từ Patchwork Monolith đến Microservices

Martin Fowler đề xuất chiến lược "**Monolith First**": bắt đầu với monolith, hiểu rõ domain qua vài iteration, rồi mới tách microservices nếu thực sự cần. Lịch sử cho thấy hầu hết hệ thống microservices thành công — Netflix, Amazon, Uber — đều **bắt đầu từ monolith**, không phải microservices ngay từ đầu.

Điều nguy hiểm nhất là cố tách một patchwork monolith trực tiếp thành microservices mà không trải qua bước modular hóa. Kết quả thường là **distributed monolith** — mang toàn bộ nhược điểm của cả monolith lẫn distributed system, không có ưu điểm nào.

Trong hệ sinh thái Spring, **Spring Modulith** (2023+) là extension chính thức của Spring Boot hỗ trợ kiến trúc *Modular Monolith*. Nó cung cấp: (1) kiểm chứng ranh giới module bằng structural verification (`ApplicationModules.verify()` — thường chạy trong test/CI), (2) integration testing từng module độc lập, và (3) event publication registry cho giao tiếp giữa các module — bước đệm tự nhiên trước khi tách microservice, đặc biệt phù hợp với chiến lược "Monolith First".

1.7. Các kịch bản áp dụng và hạn chế của Microservices

Việc chuyển đổi sang kiến trúc Microservices giống như quyết định xây thêm một cơ sở đào tạo mới. Đừng mở thêm cơ sở nếu lượng sinh viên hiện tại vẫn vừa vặn trong một tòa nhà, bởi vì chi phí vận hành, bảo vệ và quản lý nhiều cơ sở cùng lúc sẽ cao hơn rất nhiều.

1.7.1. Quyết định kiến trúc không chỉ là kỹ thuật

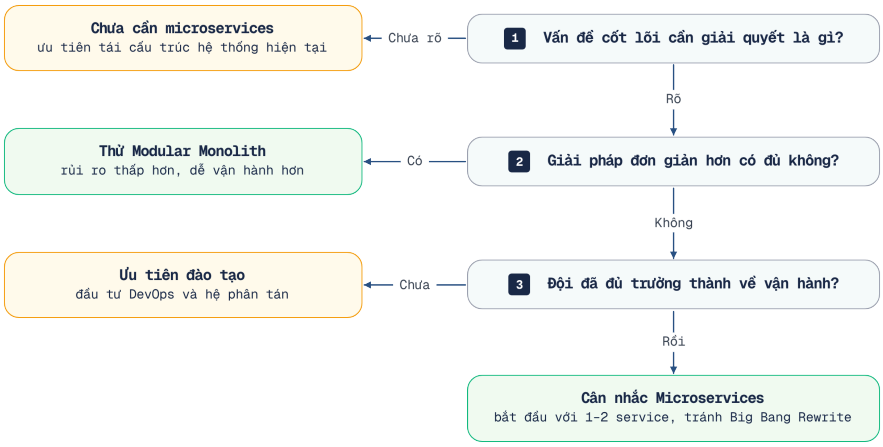
Điều quan trọng nhất cần nhấn mạnh trước khi đi vào chi tiết: **quyết định sử dụng microservices không phải thuần túy kỹ thuật**. Trong thực tế, các yếu tố phi kỹ thuật thường có ảnh hưởng lớn hơn cả yếu tố kỹ thuật:

- **Ngân sách**: Microservices đắt hơn để vận hành (infrastructure, monitoring, DevOps team)
- **Lịch trình**: Xây dựng microservices từ đầu chậm hơn monolith cho cùng feature set
- **Quy mô team**: team nhỏ thường chịu overhead quá lớn (ngưỡng ở mục Modular Monolith, §1.6)
- **Kỹ năng hiện có**: Distributed systems đòi hỏi kinh nghiệm khác biệt
- **Văn hóa tổ chức**: Các team làm việc biệt lập khó vận hành được microservices

Ngay cả Netflix (từ 2008) và Uber (từ 2014) cũng đi qua trạng thái hybrid trong lúc chuyển đổi — lựa chọn phù hợp nhất với bối cảnh lúc đó, không phải microservices thuần túy.

Trong cuốn sách này, chúng ta tập trung phân tích **khía cạnh kỹ thuật** — nhưng luôn ghi nhớ rằng đây chỉ là một chiều trong quyết định kiến trúc.

Khung ra quyết định. Ba câu hỏi cần trả lời trước khi chọn microservices:



Hình 1.7: Khung ra quyết định (Decision Framework) — ba câu hỏi trước khi chọn microservices

1.7.2. Phân tích lợi ích và chi phí

Nếu mô hình ra quyết định ở trên nghiêng về kiến trúc phân tán, bước tiếp theo là đặt lợi ích cạnh chi phí:

Bảng 1.5: Lợi ích của kiến trúc microservices

Lợi ích	Giải thích
Independent deployability	Deploy một service mà không ảnh hưởng các service khác — giảm rủi ro, tăng tần suất release
Team autonomy	Mỗi team làm chủ toàn bộ vòng đời dịch vụ — từ viết mã đến vận hành trên production
Technology diversity	Chọn công nghệ phù hợp nhất cho từng bài toán cụ thể
Selective scalability	Mở rộng riêng những service cần tài nguyên — tiết kiệm chi phí
Fault isolation	Lỗi ở một service *có thể* được cô lập, không kéo sập toàn hệ thống — với điều kiện có timeout, circuit breaker (Chương 4)

Đổi lại, những lợi ích trên không miễn phí:

Bảng 1.6: Chi phí và thách thức của microservices

Chi phí	Giải thích
Complexity tax	Hệ phân tán vốn phức tạp do độ trễ mạng, lỗi cục bộ và eventual consistency — minh chứng cho "8 Fallacies of Distributed Computing" (ví dụ: lầm tưởng rằng mạng luôn đáng tin, độ trễ bằng không)
Operational overhead	CI/CD pipeline, monitoring, logging, tracing, service discovery *cho mỗi service*
Data consistency	Transaction ACID của một database không bao phủ được nhiều service; giao dịch xuyên service thường dùng local transaction + Saga pattern, chấp nhận nhất quán kiểu BASE
Debugging difficulty	Một request đi qua 5 service → cần distributed tracing (Zipkin, Jaeger)
Integration testing	Test toàn hệ thống trở nên khó hơn nhiều bậc so với monolith

Một số thuật ngữ trong bảng trên sẽ được phân tích sâu ở các chương sau. Hiểu ngắn gọn như sau:

- **Eventual consistency** — dữ liệu giữa các service cuối cùng sẽ đồng bộ, nhưng có một khoảng trễ không cam kết trước (Chương 6).
- **ACID → BASE** — thay vì transaction "tất cả hoặc không" trên một database, hệ phân tán chấp nhận nhất quán "mềm" hơn để đổi lấy tính sẵn sàng.

- **CAP Theorem** — khi xảy ra network partition, hệ buộc phải chọn giữa nhất quán (Consistency) và sẵn sàng (Availability) (Chương 7).
- **Saga** — chuỗi giao dịch cục bộ có bước bù trừ, thay cho transaction phân tán (Chương 6).

⚠ Distributed Monolith

Nếu chia hệ thống thành nhiều "service" nhưng chúng vẫn phải deploy cùng nhau, chia sẻ database, và thay đổi đồng loạt — kết quả là một **distributed monolith**: mang toàn bộ nhược điểm của monolith *và* hệ phân tán, không có ưu điểm nào. Đây là kết quả phổ biến nhất khi tách service mà chưa modular hóa.

1.8. Case study: KBM

Quay lại với KBM. Hệ thống không bắt đầu như một kiến trúc phân tán: nó lớn lên từ vài ứng dụng nguyên khối chạy cạnh nhau, và chính chi phí bảo trì các khối đó — chứ không phải xu hướng — là lý do tách service (xem mục "Điểm khởi đầu" dưới đây).

1.8.1. Bài toán nghiệp vụ

Xuyên suốt cuốn sách này, chúng ta sử dụng một case study thực tế: **KBM** — nền tảng học tập trực tuyến (LMS) chuyên biệt cho các môn tin học thực hành, được phát triển và vận hành tại một trường đại học kỹ thuật.

Vấn đề cần giải quyết. Tại trường đại học, các môn học liên quan đến Cơ sở dữ liệu, Lập trình mạng và Quản trị hệ thống/DevOps là bắt buộc cho hàng nghìn sinh viên mỗi năm. Giảng viên không thể thực thi và kiểm tra hàng trăm bài SQL, bài lập trình mạng hoặc bài thực hành Kubernetes mỗi tuần bằng tay. Sinh viên cần phản hồi nhanh để sửa lỗi ngay; giảng viên cần môi trường thi cử công bằng; hệ thống cần cô lập dữ liệu và môi trường thực hành để bài làm của sinh viên này không ảnh hưởng sinh viên khác.

Ba nhóm người dùng chính:

- **Sinh viên**: Luyện tập SQL, tham gia thi, nộp bài tập, làm đồ án. Quy mô: Hàng nghìn / học kỳ.
- **Giảng viên**: Soạn đề, tổ chức thi/kiểm tra, chấm điểm, theo dõi tiến độ. Quy mô: Hàng chục.
- **Quản trị viên**: Quản lý tài khoản, cấu hình hệ thống, giám sát vận hành. Quy mô: Vài người.

Bốn nhóm chức năng nghiệp vụ — mỗi nhóm đặt ra yêu cầu kiến trúc khác nhau:

- **SQL Lab**: Sinh viên viết SQL trên trình soạn thảo web, nộp bài và nhận kết quả trong chế độ luyện tập (Practice Mode) hoặc chế độ thi (Contest Mode). Chế độ luyện tập ưu tiên độ trễ thấp; chế độ thi ưu tiên throughput, tính công bằng, message queue và WebSocket thời gian thực.
- **Network Lab**: Sinh viên viết Java client kết nối đến judge server qua nhiều nhóm giao thức: TCP, UDP, RMI, SOAP, REST và gRPC. Đây là nơi KBM minh họa rất rõ technology diversity: không phải mọi service đều là Spring Boot REST API.
- **Course Management**: Giảng viên tạo khóa học, bài tập cá nhân/nhóm, MCQ, điểm danh, grade scheme, đồ án tốt nghiệp và tích hợp GitHub Classroom. Yêu cầu chính: data integrity, workflow nhiều bước, cross-service query.
- **DevOps Lab**: Sinh viên thực hành Docker/Kubernetes trong lab environment cô lập, truy cập qua terminal web, được chấm bằng validator script. Phần này dùng Go, k3s và Sysbox ở mức hạ tầng khái quát — đủ để phân tích kiến trúc, không cần công khai chi tiết vận hành cụ thể.

Ràng buộc thực tế — đây là điều làm cho case study có giá trị giảng dạy:

- **Team nhỏ**: 1–2 lập trình viên chính + sinh viên hỗ trợ theo đợt, không có đội ngũ DevOps hay kiểm thử (QA) riêng
- **Hạ tầng hạn chế**: hạ tầng nhỏ, LMS chính chạy Docker Compose; DevOps Lab dùng k3s/Sysbox cho bài toán lab isolation, không công khai sơ đồ máy chủ/domain cụ thể
- **Phát triển nhanh dưới áp lực**: Hệ thống phải phục vụ sinh viên ngay, nhiều quyết định kiến trúc ưu tiên *đưa vào dùng sớm*, chấp nhận trả giá về sau
- **Hệ thống đang chạy trên production**: Mọi cải tiến phải được thực hiện tăng dần — không thể tạm dừng hoạt động để tái cấu trúc (refactor) toàn bộ

Điểm quan trọng: KBM hiện tại **không phải là hình mẫu chuẩn mực** — và đó chính là lý do nó có giá trị làm case study. KBM tách service dù đội rất nhỏ; lý do (mục đích giáo dục, nhu cầu cô lập bộ chấm) và cái giá được phân tích ở §10.1. Giống như hầu hết dự án thực tế, đội ngũ phát triển hiếm khi xây dựng được hệ thống hoàn chỉnh ngay từ đầu. Cuốn sách sẽ sử dụng KBM như **bài toán chuyển đổi thực tế**: ở mỗi chương, chúng ta phân tích hệ thống đang ở đâu, vấn đề kỹ thuật là gì, giải pháp khuyến nghị là gì, và đề xuất lộ trình cải thiện.

1.8.2. Điểm khởi đầu: những monolith rời rạc, không phải microservices

Một chi tiết dễ bị bỏ qua nhưng quyết định cả tinh thần của case study: KBM hôm nay là một hệ **polyglot microservices** (Java + Go), nhưng nó *không khởi đầu* như vậy. "Before" của KBM không phải là một hệ thống duy nhất, mà là *vài* monolith rời rạc chạy cạnh nhau:

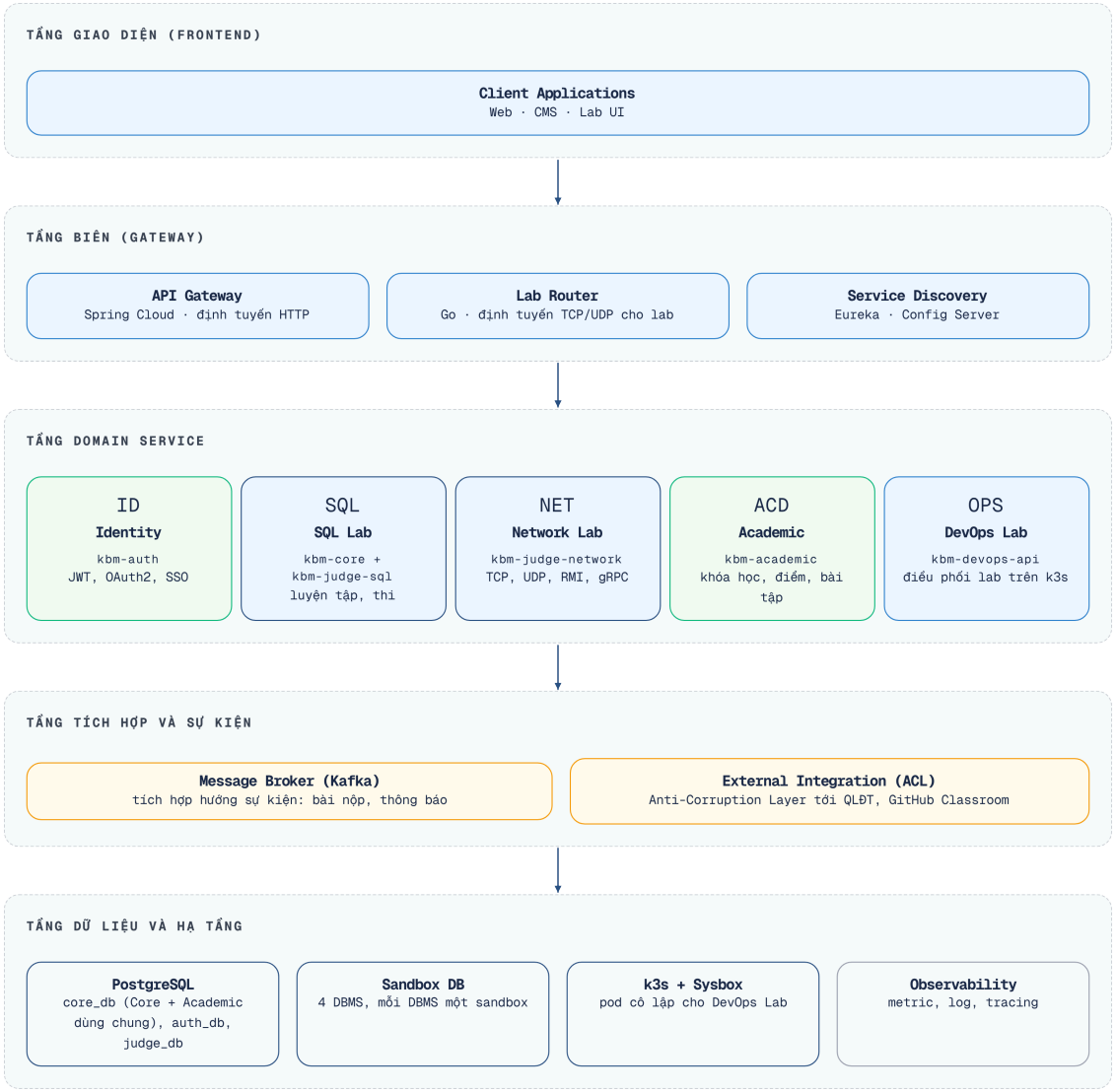
- **Quản lý đào tạo**: một ứng dụng **ASP.NET MVC** nguyên khối (`kbm-legacy-monolith`) quản lý khóa học, bài tập, điểm số, người dùng trên một database dùng chung.
- **Bộ chấm code (phiên bản đầu tiên)**: một monolith MVC **tách biệt**, do nhóm tự viết, hỗ trợ C/C++ và Java, chạy bài nộp trực tiếp trên máy chủ **không có sandbox**.
- **Bộ chấm mạng (phiên bản đầu tiên)**: lại là một monolith MVC riêng nữa cho lập trình mạng Java.

Mỗi hệ là một khối độc lập, không chia sẻ kiến trúc — nên chi phí bảo trì **nhân lên theo số hệ thống** chứ không hề được chia nhỏ. Đây mới là *động cơ thật* để tách service: một thay đổi nhỏ buộc build/deploy lại cả khối, bộ chấm không sandbox tạo rủi ro bảo mật (mã sinh viên chạy chung tiến trình máy chủ), và không thể mở rộng phần chấm độc lập với phần quản lý.

Quá trình chuyển đổi từ các monolith rời rạc sang các service `kbm-*` diễn ra **tăng dần** (Strangler Fig — xem Chương 10), không phải một lần viết lại. Bài học cốt lõi mở đầu cho cả cuốn sách: **microservices là điểm đến của một quá trình tiến hóa, không phải điểm khởi đầu** — đúng với tinh thần "Monolith First" ở §1.6. Có một "before" cụ thể như vậy giúp người đọc nhìn thấy lý do tách service, thay vì coi microservices là lựa chọn hiển nhiên.

1.8.3. Kiến trúc tổng quan

Hình 1.8 là sơ đồ giản lược kiến trúc KBM theo năm context ban đầu.



Hình 1.8: *Kiến trúc tổng quan KBM LMS (sơ đồ giản lược, xem đầy đủ ở Chương 2)*

Mô hình này không phải là trạng thái cuối cùng, mà là bối cảnh khởi đầu để chúng ta theo dõi quá trình chuyển đổi.

Sự tiến hóa của KBM qua 12 chương

KBM tiến hóa qua ba trạng thái được theo dõi xuyên suốt sách:

Trạng thái hiện tại (Chương 1–2): Các monolith ban đầu (ASP.NET quản lý đào tạo và hai bộ chấm MVC) đã được tách thành 7 bounded context core (năm context ban đầu — Identity & Access, SQL Practice, Network Practice, Academic Management, DevOps Practice — cùng Code Practice và Examination & Proctoring bổ sung khi hệ thống tiến hóa), nhưng vẫn còn shared database giữa SQL Practice và Academic Management — tạo coupling ngầm không kém gì monolith.

Trạng thái đang chuyển đổi (Chương 3–11): Mỗi chương phân tích một khoảng trống cụ thể và đề xuất lộ trình chuyển đổi: tách database, thêm Saga pattern, hoàn thiện API Gateway hiện có, triển khai Circuit Breaker... Hệ thống dần tiến về kiến trúc microservices đầy đủ.

Trạng thái mục tiêu (Chương 12): mỗi service sở hữu dữ liệu của mình — tách schema, truy cập chéo qua API, tách hẳn DB server khi có nhu cầu mở rộng riêng (Ch.7); event-driven communication qua Kafka, CI/CD pipeline tự động, observability đầu cuối.

Các bài toán kỹ thuật chính. Mỗi chương sách sẽ quay lại KBM ở góc nhìn khác nhau. Bảng 1.7 ánh xạ các bài toán kỹ thuật — từ hiện trạng đến hướng chuyển đổi:

Bảng 1.7: *Ánh xạ bài toán kỹ thuật KBM LMS theo chương*

Bài toán kỹ thuật	Hiện trạng (khảo sát đầu năm 2026)	Vấn đề cần giải quyết	Chương
Domain decomposition	7 bounded context core + supporting services	Shared library và shared DB vẫn tạo coupling	Ch.2
API design	REST controllers + Network Lab đa giao thức	Naming/versioning chưa thống nhất; contract nhiều protocol	Ch.3
Sync communication	OpenFeign, bộ điều phối `kbm-judge-sql` + các worker `kbm-sandbox-*`, tích hợp ngoài	Cần resilience + Anti-Corruption Layer rõ ràng	Ch.4
Async communication	Kafka cho chế độ thi SQL; DB queue cho DevOps Lab	Cần idempotency, retry, dead letter và workload-based choice	Ch.5
Distributed data	**Shared database** giữa SQL Practice và Academic Management	Vi phạm database-per-service, coupling cao	Ch.6, Ch.7
API Gateway	Spring Cloud Gateway + Go reverse proxy cho lab routing	Path-based vs hostname-based gateway trade-off	Ch.8
Security	JWT HS256, OAuth2, SSO cross-platform	Secret management, token storage, phòng thủ chiều sâu	Ch.9
Migration strategy	10 năm tiến hóa, legacy QLĐT, GitHub Classroom	Strangler Fig, ACL, roadmap theo rủi ro	Ch.10
Observability	Actuator/Prometheus một phần, correlation ID đang mở rộng	Cần centralized log, tracing, metric đầu cuối	Ch.11
Deployment	Docker Compose cho LMS chính; k3s/ Sysbox cho DevOps Lab	CI/CD, rollback, resource isolation, tránh thiết kế quá mức (over-engineering)	Ch.12

Trong các bài toán trên, phân chia dữ liệu là thách thức dai dẳng nhất — nhìn ngay tại KBM:

KBM sử dụng shared database giữa SQL Practice (`kbm-core`) và Academic Management (`kbm-academic`) — đây là **anti-pattern** theo nguyên tắc database-per-service [2a, Ch.2]. Hậu quả: thay đổi schema ảnh hưởng đến cả hai service, không thể mở rộng hay deploy độc lập, và hai team nếu có sẽ phải phối hợp trong mọi đợt chuyển đổi schema. Lộ trình cải thiện: tách schema → API-based data access → database-per-service. Chi tiết sẽ phân tích tại Chương 7.

Những quyết định chúng ta muốn làm lại — Cautionary Tale. Richardson trong phiên bản 2 [2b, Ch.2] dành nguyên một chương kể câu chuyện cuộc chuyển đổi FTGO thất bại — cho thấy *những sai lầm* trước khi dạy pattern đúng. KBM cũng có những bài học tương tự — năm quyết định mà team sẽ chọn khác nếu làm lại.

Năm quyết định dưới đây là để nhận diện vấn đề — cơ chế chi tiết ở các chương sau; người đọc chưa quen với JWT HS256 hay idempotency có thể lướt qua trước.

Bảng 1.8: *Năm quyết định kiến trúc KBM muốn làm lại*

#	Quyết định	Tại sao chọn lúc đó	Rủi ro & hệ quả kiến trúc	Anti-pattern tương ứng
①	**Shared database** giữa SQL Practice và Academic Management	Nhanh hơn, không cần API giữa hai service	Thay đổi schema ở một dịch vụ có thể làm lỗi dịch vụ kia nếu thiếu quy ước sở hữu bảng; không thể tách chu kỳ chuyển đổi schema độc lập, buộc phải phối hợp triển khai đồng bộ	Richardson [2b]: *Shared Database anti-pattern*
②	**Shared library** chứa quá nhiều thứ (entity, security, SQL parser)	Tái sử dụng code, giảm duplication	Thay đổi trong thư viện dùng chung thường kéo theo việc biên dịch và triển khai lại nhiều dịch vụ cùng lúc, làm mất khả năng triển khai độc lập (independent deployability)	Richardson [2b]: *Lock-step deployment*
③	**JWT HS256** (khóa đối xứng chia sẻ giữa các service)	Cấu hình nhanh chóng chỉ mất vài phút	Vì khóa dùng để xác minh (verify) cũng chính là khóa dùng để ký, nên bất kỳ dịch vụ nào bị xâm phạm (compromise) cũng đều có thể làm giả JWT. Đáng lẽ phải dùng khóa bất đối xứng (RS256)	Chi tiết tại Ch.9
④	**Không có idempotency** cho submit endpoint	"Không ai gửi yêu cầu trùng lặp đâu"	Trong chế độ thi: client tự động retry khi mạng không ổn định, dẫn đến nộp bài trùng lặp (duplicate submission) và chấm điểm sai	Chi tiết tại Ch.5, Ch.6
⑤	`docker logs` **thay cho observability**	"Quy mô nhóm nhỏ nên đọc log trực tiếp sẽ tiết kiệm thời gian hơn"	Monitoring truyền thống chỉ báo hệ thống lỗi, thiếu Observability để hiểu "tại sao". Chế độ thi: hàng trăm lượt nộp mỗi phút, Judge chậm, không xác định được bottleneck ở đâu, dẫn đến quá trình debug có thể mất hàng giờ	Chi tiết tại Ch.11

Mỗi quyết định trên đều *hợp lý tại thời điểm đó* — team nhỏ, cần phát hành sớm, ưu tiên tính năng hơn kiến trúc. Nhưng khi hệ thống mở rộng (chế độ thi, quản lý bài tập, quản lý khóa luận, Network Lab đa giao thức, DevOps Lab bằng Go), nợ kỹ thuật (technical debt) tích lũy nhanh hơn tốc độ phát triển tính năng. Đây là kiểu *penny wise and pound foolish* — tiết kiệm ở bước nhỏ, trả giá ở bước lớn. Nhưng đó cũng chính là giá trị sư phạm: **nếu hệ thống đã hoàn chỉnh từ đầu, sẽ không có gì để học** — mỗi anti-pattern là cơ hội để hiểu *tại sao* các giải pháp khuyến nghị tồn tại, không phải vì sách giáo khoa nói vậy, mà vì *hệ quả thực tế* đối với khả năng tiến hóa và vận hành hệ thống.

⚠ Sai lầm thường gặp

1. **Áp dụng microservices chỉ vì "chạy theo xu hướng" (trending)** — Nhiều team chọn microservices vì thấy Netflix, Amazon dùng, mà không tự hỏi: "Vấn đề cụ thể nào của chúng ta mà microservices giải quyết?" Hậu quả: tăng độ phức tạp mà không mang lại lợi ích. *Phòng tránh*: luôn bắt đầu từ vấn đề (Decision Framework §1.7), không từ giải pháp.
2. **Chuyển đổi đột ngột từ monolith sang microservices** — Tách patchwork monolith trực tiếp, bỏ qua bước modular hóa. Hậu quả: distributed monolith — hội tụ nhược điểm của cả hai. *Phòng tránh*: refactor ranh giới module rõ ràng *trước*, rồi mới tách service (§1.6).
3. **Tách service trước khi tổ chức team** — Chia 10 microservices nhưng vẫn 1 team duy nhất, deploy cùng nhau, review code chéo nhau. Hậu quả: overhead vận hành mà không có autonomy thực sự (Conway's Law). *Phòng tránh*: tổ chức team phù hợp *cùng lúc* hoặc *trước* khi tách service (chi tiết ở Ch.2).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **So sánh Monolith và Microservices**: monolith-vs-microservices.html

1.9. Tổng kết

Kiến trúc phần mềm đã tiến hóa từ monolith, qua SOA, đến microservices với trọng tâm đặt tính độc lập lên hàng đầu.

Thực tế cho thấy, **mỗi bước tiến hóa đều giải quyết giới hạn của bước trước, nhưng đồng thời tạo ra thách thức mới**. SOA giải phóng khỏi silo nhưng tạo ra coupling vào middleware. Microservices trao quyền tự trị cho từng service nhưng đổi lại bằng complexity tax của distributed systems. Không có kiến trúc nào là "tốt nhất" — chỉ có kiến trúc phù hợp nhất cho ngữ cảnh.

Kinh nghiệm từ Netflix, Amazon, và Uber khẳng định rằng **chuyển đổi sang microservices là phản ứng trước giới hạn thực tế, không phải theo đuổi xu hướng**. Và trạng thái hybrid — trạng thái vận hành phổ biến trong thực tế — là hoàn toàn hợp lý.

Modular monolith không phải bước lùi mà là **bước đệm thông minh**: hiểu rõ domain trước khi chia tách. Quyết định kiến trúc cũng không chỉ là kỹ thuật — ngân sách, team, lịch trình, và văn hóa tổ chức nhiều khi còn nặng hơn yếu tố kỹ thuật.

Case study KBM — một hệ thống LMS với các ràng buộc điển hình (team nhỏ, hạ tầng hạn chế) — cung cấp bài học thực tiễn về những sai lầm kiến trúc ban đầu.

1.10. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.3–4: Service-Orientation Principles, Understanding SOA
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.1: Escaping Monolithic Hell, Scale Cube
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.1: Fast Flow Success Triangle, DORA Metrics; Ch.5: Architectural Requirements for Fast Flow; Ch.6: Modular Monolith
4. [4b] Sam Newman, *Monolith to Microservices* — Ch.1: Just Enough Microservices, Should You Migrate?
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.1: SOA vs Microservices vs EDA

Sách bổ trợ:

6. Nicole Forsgren, Jez Humble, Gene Kim, *Accelerate* (2018) — DORA Metrics, nghiên cứu định lượng về software delivery performance
7. Gene Kim et al., *The DevOps Handbook*, 2nd Ed. (2021) — Three Ways of DevOps, deployment pipeline design

Nguồn trực tuyến:

- [W1] James Lewis & Martin Fowler, "Microservices" (2014) — martinfowler.com/articles/microservices.html

- [W2] Martin Fowler, “MonolithFirst” (2015) — martinfowler.com/bliki/MonolithFirst.html
- [W3] Netflix Technology Blog, “Completing the Netflix Cloud Migration” (2016)
- [W4] Uber Engineering, “Domain-Oriented Microservice Architecture” (2020)
- [W5] Steve Yegge, “Stevey’s Google Platforms Rant” (2011) — nguồn câu chuyện Bezos API Mandate
- DORA, “State of DevOps Report 2024” — dora.dev/research

Bài tập: 1-0, 1-1, 1-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[1] Erl, T. (2016). *Service-Oriented Architecture: Analysis and Design for Services and Microservices* (2nd ed.). Prentice Hall.

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O’Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.